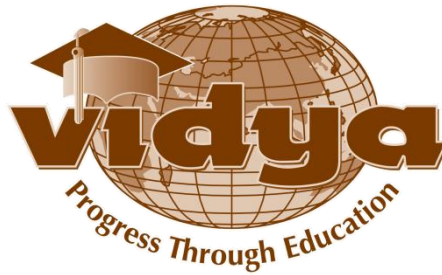


Vidya Academy of Science & Technology
(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam
Technological University Accredited by NAAC with "B++" grade



**DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING**

GXEST204 PROGRAMMING IN C

LAB MANUAL

PREPARED BY: Ms.NEETHU GEORGE

APPROVED BY:HOD, CSE

Vision of the Institution

Progress through education

Mission of the Institution

To seek, strive for and scale greater heights of quality education

M1: Seek to foster in every student a profound and practical understanding of knowledge, encouraging critical inquiry and intellectual curiosity.

M2: Strive to ensure every student develops holistically, cultivating the skills, ethical character, and resilience necessary to thrive in diverse and challenging future environments.

M3: Scale the commitment to service by inspiring students to apply their knowledge and skills to actively address community needs, promoting ethical leadership and social responsibility.

Vision of the Department

Progress through quality education and become a centre of excellence in computing knowledge

Mission of the Department

M1: To seek and strive for advancements in technological domains.

M2: To create socially responsible engineers in the field of computer science.

M3: To bridge the gap between science and mankind through meaningful innovation and service.

Program Educational Objectives (PEOs)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

- PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.
- PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAMMING IN C

(Common to Group A & B)

Course Code	GXEST204	CIE Marks	40
Teaching Hours/Week(L: T:P: R)	3:0:2:0	ESE Marks	60
Credits	4	Exam Hours	2 Hrs. 30 Min.
Prerequisites (if any)	None	Course Type	Theory

Course Objectives:

1. To prepare learners to write versatile C programs for solving computational problems that they come across in their professional life.
2. To equip the learner to write efficient C programs using suitable language constructs to solve real world computational problems.

SYLLABUS

Module No.	Syllabus Description	Contact Hours
1	C Fundamentals - Character Set, Constants, Identifiers, Keywords, Basic Data types, Variables, Operators and its precedence, Bit-wise operators, Expressions; Statements - Input and Output statements; Structure of a C program; Simple programs. Control Statements - if, if-else, nested if, switch, while, do-while, for, break & continue, nested loops.	9
2	Arrays - Single dimensional arrays, Defining an array, Array initialization, Accessing array elements; Enumerated data type; Type Definition; Two-dimensional arrays – Defining a two-dimensional array; Programs for matrix processing; Programs for sequential search; Bubble sort; Strings - Declaring a string variable, Reading and displaying strings, String related library functions – Programs for string matching.	9

3	Functions - Function definition, Function call, Function prototype, Parameter passing; Recursion; Passing array to function; Macros - Defining and calling macros; Command line Arguments. Structures - Defining a Structure variable, Accessing members, Array of structures, Passing structure to function; Union. Storage Class - Storage Classes associated with variables: automatic, static, external and register.	9
4	Pointers - Declaration, Operations on pointers, Passing pointer to a function, Accessing array elements using pointers, Processing strings using pointers, Pointer to pointer, Array of pointers, Pointer to function, Pointer to structure, Dynamic Memory Allocation. Files- Different types of files in C, Opening & Closing a file, Writing to and Reading from a file, Processing files, Library functions related to file – fseek(), ftell(), fread(), fwrite().	9

Course Assessment Method (CIE: 40 marks, Continuous Internal Evaluation Marks (CIE):

Attendance	Continuous Assessment (Lab)	Internal Examination-1 (Written Examination)	Internal Examination-2 (Written Examination)	Internal Examination- 3 (Lab Examination)	Total
5	5	10	10	10	40

End Semester Examination Marks (ESE)

In Part A, all questions need to be answered and in Part B, each student can choose any one full question out of two questions

Part A	Part B	Total
2 Questions from each module. - Total of 8 Questions, each carrying 3 marks (8x3 =24marks)	- Each question carries 9 marks. - Two questions will be given from each module, out of which 1 question should be answered. - Each question can have a maximum of 3 sub divisions. (4x9 = 36 marks)	60

Course Outcomes (COs)

At the end of the course students should be able to:

Course Outcome		Bloom's Knowledge Level (KL)
CO1	Infer a computational problem and develop C programs from them using basic constructs of C language including the control statements.	K2
CO2	Develop C programs using arrays, matrices, and strings.	K3
CO3	Utilize functions to find solution to the computational problems by dividing it into a number of modules and abstract data types.	K3
CO4	Develop C programs using pointers for dynamic data handling.	K3
CO5	Use files in C to permanently store and manipulate data.	K3

Note: K1- Remember, K2- Understand, K3- Apply, K4- Analyse, K5- Evaluate, K6- Create

CO-PO Mapping Table:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO1 0	PO1 1	PO12
CO1	3	3	3	3	-	-	-	-	-	-	-	3
CO2	3	3	3	3	-	-	-	-	-	-	-	3
CO3	3	3	3	3	-	-	-	-	-	-	-	3
CO4	3	3	3	3	-	-	-	-	-	-	-	3
CO5	3	3	3	3	-	-	-	-	-	-	-	3

Text Books				
Sl. No	Title of the Book	Name of the Author/s	Name of the Publisher	Edition and Year
1	Programming with C	Byron S Gottfried	Mc Graw Hill	4/e, 2018
2	Problem Solving and Program Design in C	Jeri R. Hanly, Elliot B. Koffman	Pearson	8/e, 2016

Reference Books				
Sl. No	Title of the Book	Name of the Author/s	Name of the Publisher	Edition and Year
1	The C Programming Language	Brian W. Kernighan and Dennis Ritchie	Pearson	2/e, 2015
2	C The Complete Reference	Herbert Schildt	Mc Graw Hill	4/e, 2017
3	Let us C	Yashavant Kanetkar	BPB Publishers	19/e, 2022
4	Programming in ANSI C	E Balagurusamy	Mc Graw Hill	9/e, 2024

Lab Setup Preparation

Before you begin programming, ensure your environment is correctly set up.

1. Install an Integrated Development Environment (IDE) or Text Editor & Compiler

You need two main components: a place to **write** your code and a tool to **translate** it.

- **IDE (Recommended for beginners):** An IDE combines a text editor, compiler, and debugger into a single application.
 - **Examples:** Code::Blocks, Visual Studio Code (with C/C++ extensions), or Dev-C++.
- **Text Editor & Compiler (for advanced users):**
 - **Text Editor Examples:** Sublime Text, Notepad++, or Vim.
 - **Compiler:** The most common C compiler is **GCC (GNU Compiler Collection)**. On Windows, this is often installed as part of the **MinGW** package.

2. Verify the Compiler Path

Make sure your IDE or command line can find the C compiler.

- **IDE:** Most IDEs automatically configure the compiler path upon installation. Check the settings (e.g., *Settings > Compiler > Executables*).
- **Command Line (Windows/Linux/macOS):** Open your terminal or command prompt and type:
- Bash

`gcc --version`

- If it displays the compiler version, you're ready. If it fails, you need to add the compiler's installation directory (e.g., the bin folder of MinGW) to your system's **PATH environment variable**.

General Lab Instructions

Follow these guidelines for writing and submitting your C programs.

1. Coding Conventions

- **Include Necessary Headers:** Always include the header file for functions you use (e.g., `#include <stdio.h>` for `printf`).
- **Use Descriptive Variable Names:** Choose names that clearly indicate the variable's purpose (e.g., `student_score` instead of `s`).
- **Indent Properly:** Use consistent indentation (4 spaces or 1 tab) to make the code structure clear.
- **Use Comments:**
 - Explain **complex logic** or **sections** of the code using `//` for single-line comments or `/* ... */` for multi-line comments.
 - Include a **comment block** at the top of every file with your **Name, ID, Date, and Program Description**.

2. Program Development Cycle

1. **Write:** Type your code in the text editor or IDE.
2. **Save:** Save your file with the **.c extension** (e.g., `lab1_hello.c`).
3. **Compile:** Use the IDE's "Build" function or the command line:
4. Bash

```
gcc your_file_name.c -o your_program_name
```

5. **Fix Syntax Errors:** If the compiler reports errors, **read the error messages carefully** (they usually indicate the line number) and fix them.
6. **Run/Execute:** Run the compiled executable file:
 - **IDE:** Use the "Run" button.
 - **Command Line:** `./your_program_name` (Linux/macOS) or `your_program_name.exe` (Windows).
7. **Test and Debug:** Test your program with various inputs to ensure it produces the **correct output**. If the output is wrong (a **logical error**), use the debugger or print statements to find the mistake.

3. Submission Requirements

- **File Naming:** Adhere to the instructor's specified naming convention (e.g., `StudentID_LabX_ProgramY.c`).
- **Source Code Only:** Submit the **.c source file** unless otherwise instructed. **Do not** submit the compiled executable file (.exe or equivalent).
- **Clear Output:** Ensure your program's output is clean and easy to read for the grader.

INDEX

Exp No.	Expt. Name	Page Number
1	Familiarization of Hardware Components of a Computer	12
2	Familiarization of Linux environment – Programming in C with Linux	16
3	Familiarization of Console I/O and Operators In C	19
4	Evaluate the arithmetic expression $((a - b / c * d + e) * (f + g))$ and display its solution.	23
5	Read 3 integer values, find the largest among them.	25
6	Read a natural number and check whether the number is prime or not	28
7	Read a natural number and check whether the number is armstrong or not	32
8	Read n integers, store them in an array and find their sum and average	36
9	Program to read n integers, store them in an array and search for an element in the array using the linear search algorithm	39
10	Bubble sort implementation of n integers	43
11	Program to check whether a given string is a palindrome or not	46
12	Concatenate two strings without using library functions	49
13	Read a string ending with a \$ symbol, store it in an array, and count the vowels, consonants, and spaces	52
14	Addition of two distances using structures in euclidean space	55
15	Reading and printing data of n employees using structure	58
16	Declare a union with 5 string variables and display address	62
17	Factorial of a natural number using functions in c	66
18	Reverse a string using a user defined function	69

19	Program for matrix operations using functions	72
20	Program to add and swap two numbers using pointers	78
21	Input and print the elements of an array using pointer	81
22	Compute sum of the elements stored in an array using pointers and user defined functions.	83
23	Create a file, write data, read and display file contents, append new data, and display the updated file contents using c file handling	87
24	Count the number of characters, words, and lines in a text file	91

EXP NO:1**FAMILIARIZATION OF HARDWARE COMPONENTS OF A COMPUTER****AIM:-**

To identify, locate, describe the function of, and understand the interconnections between the major internal and external hardware components that make up a modern computer system.

LEARNING OUTCOMES:-

Upon successful completion of this lab, the student will be able to:

- Differentiate between internal (System Unit) and external (Peripheral) hardware components.
- Identify and locate the major internal components (CPU, Motherboard, RAM, HDD/SSD, Power Supply) inside a computer case.
- Explain the primary function of each major component and its role in the overall computer architecture.
- Trace the path of data and power between key components (e.g., CPU to RAM, Motherboard to Power Supply).
- Demonstrate proper handling procedures for delicate components like RAM and expansion cards.

SOFTWARE REQUIRED:-

Since this is a hardware familiarization lab, minimal software is required.

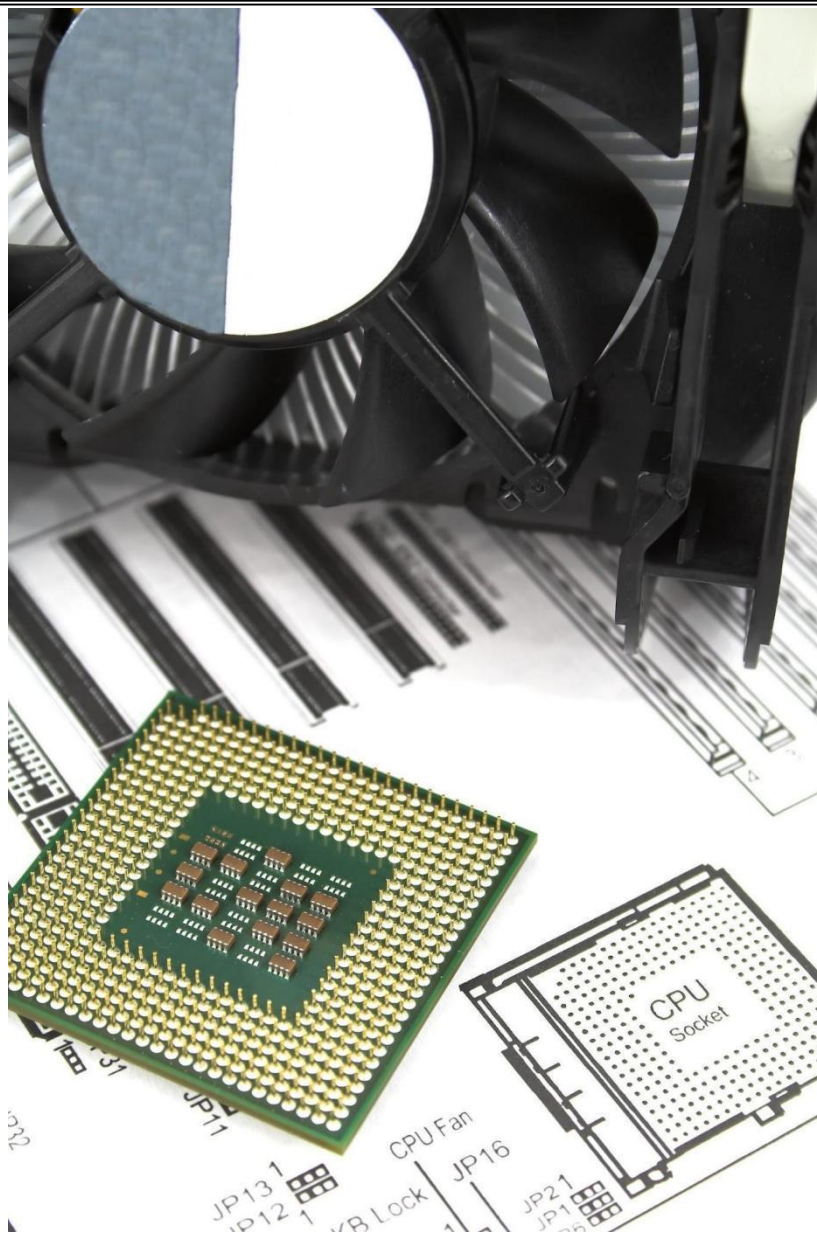
- Operating System (OS): Any standard OS (Windows, Linux, or macOS) on the lab computers to observe the system's operational state.
- System Information Utility (Optional): Tools like CPU-Z, Speccy (Windows), or system information viewers (built into the OS) to verify installed hardware specifications digitally.

DESCRIPTION:-

A computer system is fundamentally divided into two major parts: Hardware and Software. Hardware is the collection of physical parts that constitute a computer system, while software is the set of instructions that the hardware executes. The architecture is based on the Von Neumann Model, where a single memory space is used for both instructions and data.

The main internal components and their relationship are key:

1. Central Processing Unit (CPU): The "brain" of the computer. It executes instructions, performs calculations, and manages the flow of data.



Shutterstock

2. Random Access Memory (RAM): The temporary, volatile memory where the OS, applications, and current data are loaded for quick access by the CPU.
3. Motherboard: The main circuit board that acts as the communication backbone, providing sockets for the CPU, slots for RAM, connections for storage, and expansion slots for peripherals.
4. Storage Devices (HDD/SSD): Non-volatile memory used for permanent storage of the OS, applications, and user data.
5. Power Supply Unit (PSU): Converts AC power from the wall outlet into the regulated DC power required by all internal components.

ALGORITHM:-

The procedure outlines the steps to physically examine and document the hardware components.

1. Preparation and Safety:
 - Ensure the lab computer is powered off and unplugged from the wall socket.
 - Don anti-static wrist straps and attach them to a grounded metallic part of the computer case to prevent Electrostatic Discharge (ESD) damage.
2. Case Access:
 - Carefully open the side panel of the computer case (usually secured with thumbscrews or latches).
3. Component Identification (Internal):
 - Locate the Motherboard: Identify the large green/blue circuit board that fills the bottom of the case.
 - Identify the CPU: Locate the large fan/heatsink assembly typically near the center of the motherboard. Do not remove the heatsink.
 - Locate RAM: Identify the long, thin sticks plugged into the specialized slots (DIMM slots) on the motherboard.
 - Locate Storage: Locate the hard drives (HDD/SSD) and note how they are mounted and connected to the motherboard (usually via SATA cables) and the PSU.
 - Identify the PSU: Locate the metal box, usually at the top or bottom rear of the case, with many thick cables originating from it.
4. Component Identification (External):
 - Examine the Input/Output (I/O) Panel at the back of the case.
 - Identify and document the various ports: USB (different types/generations), Ethernet (RJ-45), Audio jacks, and Video ports (HDMI, DisplayPort, VGA).
5. Documentation:
 - Record the type, location, and a brief description of the function for each component identified in a structured table format.
6. Reassembly and Verification:
 - Carefully close and secure the computer case panel.
 - Plug in the power cord and peripherals, then boot the computer.
 - Run the System Information Utility (if applicable) to verify that the observed components match the reported specifications.

MODEL VIVA QUESTIONS:-

These questions are designed to test the student's understanding of the functions and relationships between the hardware components examined in the lab.

1. Function and Role

- What is the primary function of the Motherboard in a computer system?
- Explain the difference between RAM (Random Access Memory) and HDD/SSD (Hard Drive/Solid State Drive) in terms of data volatility and speed.
- What is the role of the Power Supply Unit (PSU)? Why is wattage important?
- If the computer constantly freezes during intense tasks, which two components are most likely being stressed, and how might they be cooled? (Hint: CPU and GPU)
- What is the significance of the Expansion Slots (e.g., PCIe slots) on the Motherboard?

2. Terminology and Identification

- Define ESD (Electrostatic Discharge) and explain why an anti-static wrist strap is mandatory for this lab.
- What is a Bus in the context of a motherboard? Name one type of bus found on the board.
- Identify the common cable used to connect storage devices (HDD/SSD) to the Motherboard. (Answer: SATA)

3. Application and Troubleshooting

- A user wants to increase the speed of running multiple applications simultaneously. Which component should they upgrade?
- If a computer powers on, but nothing appears on the screen, what internal component is the first to check after the display cable connection?
- Describe the process by which the CPU communicates with data stored temporarily in RAM.

RESULT:-

Familiarization with each component helps in understanding the overall functioning of a computer system.

EXP NO.2**FAMILIARIZATION OF LINUX ENVIRONMENT – PROGRAMMING IN C WITH LINUX****AIM:-**

To familiarize with the Linux operating system environment and to develop programming skills in C using Linux tools, commands, and system utilities .

LEARNING OUTCOMES:-

After completing this experiment, the student will be able to:

Navigate the Linux file system using terminal commands.

Create and edit C source files in Linux.

Compile C programs using GCC compiler.

Execute C programs and analyze outputs.

Understand the interaction between Linux OS and C programming.

DESCRIPTION:-

Linux is an open-source, multi-user, multitasking operating system widely used for system programming and application development. C is a powerful, efficient programming language that works closely with system hardware, making it ideal for Linux-based programming.

In Linux, C programs are:

1. Written using a text editor.
2. Saved with .c extension.
3. Compiled using gcc.
4. Executed using the terminal.

Basic Linux Commands Used

Command	Description
pwd	Shows current directory
ls	Lists files and directories

cd	Changes directory
mkdir	Creates directory
touch	Creates file
gcc	Compiles C program
./a.out	Executes compiled program

ALGORITHM:-

1. Start the Linux system.
2. Open the terminal.
3. Create a C source file using a text editor.
4. Write a simple C program.
5. Save the file with .c extension.
6. Compile the program using GCC.
7. Execute the program.
8. Observe the output.

PROGRAM:-

Sample C Program to print a message

```
#include <stdio.h>
int main()
{
    printf("Welcome to C Programming in Linux!\n");
    return 0;
}
```

PROCEDURE:-

Boot the system into Linux OS.

Open the terminal.

Create a new file

Type the C program and save the file.

Compile the program using:

```
gcc sample.c
```

Execute the program using:

```
./a.out
```

observe the output displayed on the screen

OUTPUT:-

Welcome to C Programming in Linux!

MODEL VIVA QUESTIONS:-

1. What is Linux?
2. What is GCC?
3. Why is C widely used in Linux?
4. What is the use of gcc command?
5. What does ./a.out indicate?
6. Difference between Linux and Windows?
7. What is open-source software?
8. What is the role of the terminal in Linux?

RESULT:-

Linux provides an efficient environment for system-level programming. GCC compiler successfully compiles and executes C programs. C programming integrates smoothly with Linux OS.

EXP.NO:3**FAMILIARIZATION OF CONSOLE I/O AND OPERATORS IN C****AIM:-**

- To understand the use of console input and output functions in C.
- To learn the use of arithmetic operators in C programming.
- To write and execute simple C programs using printf() and scanf().

LEARNING OUTCOMES:-

After completing this experiment, the student will be able to:

1. Use console I/O functions printf() and scanf().
2. Apply arithmetic operators like + and *.
3. Read input from the user through the keyboard.
4. Display output on the console.
5. Develop simple C programs using variables and operators.

DESCRIPTION:-

C provides standard input and output functions through the `stdio.h` header file.

Console I/O Functions

- printf() → Used to display output on the screen.
- scanf() → Used to read input from the user.
- Operators Used
 - + → Addition
 - * → Multiplication

These operators help perform mathematical calculations in C programs.

ALGORITHM:-

(i) Display “Hello World”

1. Start
2. Print “Hello World”
3. Stop

(ii) Read two numbers, add them and display their sum

1. Start
2. Declare two integer variables
3. Read the two numbers
4. Add the numbers
5. Display the sum
6. Stop

(iii) Read the radius of a circle and display its area

1. Start
2. Declare radius and area variables
3. Read the radius
4. Calculate area using formula:
$$\text{Area} = \pi \times r \times r$$
5. Display the area
6. Stop

PROGRAM:-

(i) Program to Display “Hello World”

```
#include <stdio.h>
```

```
int main()
{
    printf("Hello World");
    return 0;
}
```

(ii) Program to Add Two Numbers

```
#include <stdio.h>
```

```
int main()
{
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d", sum);
    return 0;
}
```

(iii) Program to Calculate Area of a Circle

```
#include <stdio.h>
int main()
{
    float radius, area;
    float pi = 3.14;

    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);

    area = pi * radius * radius;

    printf("Area of the circle = %.2f", area);
    return 0;
}
```

PROCEDURE:-

1. Open the C compiler.
2. Create a new C source file.
3. Type the program code.
4. Save the file with .c extension.
5. Compile the program.
6. Run the program.
7. Enter input values when prompted.
8. Observe the output on the screen.

OUTPUT:-

(i) Hello World

Hello World

ii) Addition of Two Numbers

Enter two numbers: 5 10

Sum = 15

(iii) Area of a Circle

Enter the radius of the circle: 7

Area of the circle = 153.94

MODEL VIVA QUESTIONS:-

1. What is console input and output?
2. What is the use of `stdio.h`?
3. Difference between `printf()` and `scanf()`?
4. What are arithmetic operators?
5. What data type is used for decimal values?
6. Why is `&` used in `scanf()`?
7. What is the value of π used in the program?
8. What is the format specifier for `int` and `float`?

RESULT:-

Console I/O functions allow interaction between user and program. Arithmetic operators are used to perform calculations. C programs can efficiently handle user input and output operations.

EXP.NO:4**EVALUATE THE ARITHMETIC EXPRESSION ((A -B / C * D + E) * (F +G))
AND DISPLAY ITS SOLUTION.****AIM:-**

To evaluate the arithmetic expression $((a - b / c * d + e) * (f + g))$ and display the result.

LEARNING OUTCOMES:-

After completing this exercise, learners will be able to:

1. Understand the order of operations in arithmetic expressions.
2. Implement arithmetic computations in a programming language.
3. Write algorithms and code to solve mathematical problems.
4. Debug and verify results using sample inputs.

DESCRIPTION:-

In arithmetic expressions:

- 1) Operator precedence is important:
 - i) () parentheses
 - ii) * multiplication, / division
 - iii) + addition, - subtraction
- 2) Expressions are evaluated based on BODMAS (Brackets, Orders, Division/Multiplication, Addition/Subtraction).
- 3) In the expression $((a - b / c * d + e) * (f + g))$, first the division and multiplication are evaluated, then subtraction/addition inside the parentheses, and finally multiplication between the parentheses.

ALGORITHM:-

1. Start.
2. Read input values for a, b, c, d, e, f, and g.
3. Compute the inner expression: $inner = a - (b / c * d) + e$.
4. Compute the second parenthesis: $second = f + g$.
5. Compute the final result: $result = inner * second$.
6. Display the result.
7. Stop.

PROGRAM:-

```
#include <stdio.h>
int main()
{
    float a,b,c,d,e,f,g,exp;
    printf("Enter the values a b c d e f g in order\n");
    scanf("%f%f%f%f%f%f%f",&a,&b,&c,&d,&e,&f,&g);
    exp = ((a - b / c * d + e) * (f + g));
    printf("Expression value = %f\n", exp);
    return 0;
}
```

PROCEDURE:-

Open the Linux terminal.

Create a new C source file using a text editor (for example, vi, nano, or gedit).

Include the required header file `stdio.h` at the beginning of the program.

Declare the variables `a`, `b`, `c`, `d`, `e`, `f`, `g` and `exp` as floating-point values.

In the `main()` function, clear the screen using the `system("clear")` command.

Prompt the user to enter the values of the variables.

Read the input values using the `scanf()` function.

Evaluate the arithmetic expression $((a - b / c * d + e) * (f + g))$ and store the result in the variable `exp`.

Display the computed result using the `printf()` function.

Save the program and compile it using the GCC compiler.

Execute the compiled program and observe the output.

OUTPUT:-

Enter the values a b c d e f g in order

10 2 5 4 6 3 7

Expression value = 68.400000

MODEL VIVA QUESTIONS:-

1. What is operator precedence in C?
2. Why is the expression evaluated using floating-point variables?
3. What data type is used to store real numbers in C?
4. Why are parentheses used in the given arithmetic expression?
5. What happens if `g` is not initialized or read from input?

RESULT:-

Evaluated expression.

EXP.NO:5**READ 3 INTEGER VALUES, FIND THE LARGEST AMONG THEM.****AIM:-**

To read three integer values and determine the largest among them.

LEARNING OUTCOMES:-

1. By the end of this program, students will be able to:
2. Understand the concept of conditional statements (like `if-else`) in C.
3. Learn how to compare multiple values.
4. Gain experience in taking input from the user.
5. Understand how to output based on conditions.

DESCRIPTION:-

This C program reads **three integer values** entered by the user, compares them, and outputs the largest value among them. The program utilizes conditional statements (`if-else`) to compare the values and determine the largest.

ALGORITHM:-

1. Start
2. Declare three integer variables `a`, `b`, and `c`.
3. Prompt the user to enter three integer values.
4. Read the input values into the variables.
5. Compare the values:
6. If `a` is greater than both `b` and `c`, then `a` is the largest.
7. Otherwise, if `b` is greater than both `a` and `c`, then `b` is the largest.
8. If neither of the above conditions is true, then `c` is the largest.
9. Display the largest value.
10. End.

PROGRAM:-

```
#include <stdio.h>
```

```
int main() {  
    int a, b, c;  
    // Taking input for 3 integers  
    printf("Enter three integers: ");  
    scanf("%d %d %d", &a, &b, &c);
```

```
// Finding the largest among the three
if (a >= b && a >= c) {
    printf("The largest number is: %d\n", a);
} else if (b >= a && b >= c) {
    printf("The largest number is: %d\n", b);
} else {
    printf("The largest number is: %d\n", c);
}

return 0;
}
```

PROCEDURE:-

Compile and Run the Program:

- Save the program in a file, for example, largest_number.c.
- Open a terminal (or command prompt), navigate to the file's directory, and
- compile the code using a C compiler like gcc:
 - gcc largest_number.c -o largest_number

Run the compiled program:

./largest_number

- Enter three integers when prompted by the program.
- The program will output the largest value among the three integers.

OUTPUT:-

Enter three integers: 10 25 30

The largest number is: 30

MODEL VIVA QUESTIONS:-

1. The program uses `if-else` conditions to compare the integers and prints the largest.
2. What is the significance of the `>=` operator in the conditions?
3. The `>=` operator is used to check if one number is greater than or equal to the other numbers, ensuring that the largest number is chosen correctly.
4. Can this program be modified to find the smallest number?
5. Yes, you can change the comparison logic to find the smallest number instead of the largest.
6. What will happen if all the numbers are equal?
7. In that case, any of the numbers will be printed as the largest, as they are all the same.

RESULT:-

The program efficiently takes three integer inputs, compares them using simple conditional statements, and prints the largest value. It showcases basic input/output operations and conditional logic in C programming.

EXP.NO:6**READ A NATURAL NUMBER AND CHECK WHETHER THE NUMBER IS PRIME OR NOT****AIM:-**

To read a natural number and check whether the number is prime or not.

LEARNING OUTCOMES:-

1. Understand the concept of prime numbers.
2. Learn how to use loops (for checking divisibility) and conditional statements (if-else) to determine if a number is prime.
3. Gain experience with input/output operations in C.
4. Understand how to handle natural numbers in programming.

DESCRIPTION:-

This C program reads a natural number entered by the user, checks whether the number is prime, and outputs the result. A prime number is a natural number greater than 1 that has no divisors other than 1 and itself.

The program checks if the number is divisible by any number from 2 to the square root of the number. If it is divisible by any of these, it is not prime; otherwise, it is prime.

ALGORITHM:-

1. Start
2. Declare a variable `n` for the input number and a variable `isPrime` to store the result.
3. Prompt the user to enter a natural number.
4. Read the integer value into `n`.
5. If `n <= 1`, then it is not a prime number.
6. Otherwise, check if the number is divisible by any integer from 2 to $\sqrt{n}$:
7. If the number is divisible by any number in this range, then it is not prime.
8. If no divisors are found, then the number is prime.
9. Display the result (Prime or Not Prime).
10. End

PROGRAM:-

```
#include <stdio.h>
#include <math.h>

int main() {
    int n, i, isPrime = 1;

    // Taking input for the natural number
    printf("Enter a natural number: ");
    scanf("%d", &n);

    // Edge case: Numbers less than or equal to 1 are not prime
    if (n <= 1) {
        isPrime = 0;
    }

    // Check for factors from 2 to sqrt(n)
    for (i = 2; i <= sqrt(n); i++) {
        if (n % i == 0) {
            isPrime = 0;
            break;
        }
    }

    // Output the result
    if (isPrime) {
        printf("%d is a Prime number.\n", n);
    } else {
        printf("%d is not a Prime number.\n", n);
    }

    return 0;
}
```

PROCEDURE:-

Compile and Run the Program:

- Save the program as check_prime.c.
- Open the terminal or command prompt and navigate to the folder where the file is saved.
- Compile the program using a C compiler:

```
gcc check_prime.c -o check_prime
```

Run the compiled program:

```
./check_prime
```

- Enter a natural number when prompted.
The program will output whether the number is prime or not.

OUTPUT:-

Enter a natural number: 7

7 is a Prime number.

Enter a natural number: 15

15 is not a Prime number.

MODEL VIVA QUESTIONS:-**What is a prime number?**

A prime number is a natural number greater than 1 that has no divisors other than 1 and itself.

Why do we check divisibility up to the square root of the number?

Any factor of a number n will always have a corresponding factor less than or equal to $\sqrt{n}$. So, checking divisibility beyond $\sqrt{n}$ is unnecessary.

What is the role of the isPrime variable?

The isPrime variable is used to track whether the number is prime (1) or not (0). It starts as 1 (assuming the number is prime) and is set to 0 if a divisor is found.

Can this program be modified to check for prime numbers up to a certain limit?

Yes, you can modify the program to loop over a range of numbers and check each one for primality.

What happens if the user enters a negative number or zero?

In the current program, if the user enters a number less than or equal to 1, it is automatically classified as "Not Prime."

What is the time complexity of the prime-checking algorithm?

The time complexity of the algorithm is $O(\sqrt{n})$, because we only check divisibility up to the square root of the number.

RESULT:-

The program executed and output obtained.

EXP.NO:7**READ A NATURAL NUMBER AND CHECK WHETHER THE NUMBER IS ARMSTRONG OR NOT****AIM:-**

To read a natural number and check whether the number is an Armstrong number or not.

LEARNING OUTCOMES:-

By the end of this program, students will be able to:

1. Understand the concept of an Armstrong number (also known as a Narcissistic number).
2. Learn how to extract digits of a number and raise them to the power of the number of digits.
3. Implement the Mathematical operation in C to check if a number is Armstrong.
4. Gain experience with loops, input/output operations, and conditional statements in C.

DESCRIPTION:-

An Armstrong number (or Narcissistic number) is a number that is equal to the sum of its own digits each raised to the power of the number of digits.

For example:

- 153 is an Armstrong number because:
 $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$
- 370 is another example:
 $3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370$

This program reads a natural number from the user, calculates if the sum of its digits raised to the power of the number of digits equals the number itself, and prints whether it is an Armstrong number or not.

ALGORITHM:-

1. Start
2. Declare the integer variable n, originalNumber, sum = 0, digit, and count for the number of digits.
3. Read the input value for n.
4. Copy the value of n to originalNumber.
5. Count the number of digits in n by dividing it by 10 repeatedly.
6. Use a loop to extract each digit from n, raise it to the power of the number of digits, and add it to the sum.

7. After processing all digits, compare the sum with the original number:
8. If the sum equals the original number, it is an Armstrong number.
9. Otherwise, it is not an Armstrong number.
10. Display the result.
11. End

PROGRAM:-

```
#include <stdio.h>

#include <math.h>

int main() {

    int num, originalNum, remainder, digits = 0;

    int result = 0;

    printf("Enter a natural number: ");

    scanf("%d", &num);

    originalNum = num;

    // Count number of digits

    while (originalNum != 0) {

        originalNum /= 10;

        digits++;

    }

    originalNum = num;

    // Calculate Armstrong sum

    while (originalNum != 0) {

        remainder = originalNum % 10;

        result += pow(remainder, digits);

        originalNum /= 10;

    }

    // Check result

    if (result == num)

        printf("%d is an Armstrong number.\n", num);
```

```
else  
  
    printf("%d is not an Armstrong number.\n", num);  
  
    return 0;  
  
}
```

PROCEDURE:-

Start the program.

Declare the required variables to store the number, original number, remainder, digit count, and result.

Read the natural number from the user.

Store the entered number in a temporary variable.

Count the total number of digits in the number using a `while` loop.

Reset the temporary variable to the original number.

Extract each digit using the modulus operator (%).

Raise each extracted digit to the power of the total number of digits using the `pow( )` function.

Add the obtained values to calculate the Armstrong sum.

Compare the calculated sum with the original number.

If both are equal, display that the number is an Armstrong number; otherwise, display that it is not an Armstrong number.

Stop the program

OUTPUT:-

Enter a natural number: 153

153 is an Armstrong number.

Enter a natural number: 125

125 is not an Armstrong number.

MODEL VIVA QUESTIONS:-

What is an Armstrong number?

An Armstrong number is a number that is equal to the sum of its digits raised to the power of the total number of digits.

2. Give an example of an Armstrong number.

153, because $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$.

3. Why is `pow()` function used in the program?

To raise each digit to the power of the number of digits.

4. Which header file is required for pow()?
math.h

5. What does % operator do?

It returns the remainder of a division and is used to extract digits.

6. Why is the original number stored in another variable?

Because the number is modified during digit extraction and needs to be preserved for comparison.

7. What type of loop is used in this program?

while loop.

8. Can this program work for any number of digits?

Yes, it works for any natural number.

RESULT:-

The C program to check whether a given natural number is an Armstrong number or not was written, compiled, and executed successfully.

EXP.NO:8**READ N INTEGERS, STORE THEM IN AN ARRAY AND FIND THEIR SUM AND AVERAGE****AIM:-**

To write a C program to read n integers, store them in an array, and find their sum and average.

LEARNING OUTCOMES:-

Understand the concept of arrays in C .

Read multiple values using loops

Perform arithmetic operations like sum and average

Use for loops effectively

Display calculated results correctly.

DESCRIPTION:-

An array is a collection of elements of the same data type stored in contiguous memory locations.

In this program, the user enters the number of elements (n) and then enters n integers. These integers are stored in an array. A loop is used to calculate the sum of all elements, and the average is obtained by dividing the sum by n

ALGORITHM:-

1. Start the program.
2. Read the value of n.
3. Declare an array of size n.
4. Read n integers and store them in the array.
5. Initialize sum to 0.
6. Add each array element to sum using a loop.
7. Calculate average as $\text{average} = \text{sum} / n$.
8. Display the sum and average.
9. Stop the program.

PROGRAM:-

```
#include <stdio.h>

int main() {

    int n, i;

    int arr[100];

    int sum = 0;

    float average;

    printf("Enter the number of elements: ");

    scanf("%d", &n);

    printf("Enter %d integers:\n", n);

    for (i = 0; i < n; i++) {

        scanf("%d", &arr[i]);

        sum += arr[i];

    }

    average = (float)sum / n;

    printf("Sum = %d\n", sum);

    printf("Average = %.2f\n", average);

    return 0;

}
```

PROCEDURE:-

1. Start the program.
2. Declare variables and an integer array.
3. Read the number of elements (n).
4. Read n integers and store them in the array.
5. Add each element to find the sum.
6. Calculate the average using the formula sum / n .
7. Display the sum and average.
8. End the program.

OUTPUT:-

Enter the number of elements: 5

Enter 5 integers:

10 20 30 40 50

Sum = 150

Average = 30.00

MODEL VIVA QUESTIONS:-

1. What is an array?
An array is a collection of elements of the same data type stored in continuous memory locations.
2. Why is an array used in this program?
To store multiple integer values efficiently.
3. What loop is used in this program?
for loop.
4. Why is average stored as a float?
Because the average may be a fractional value.
5. What is the formula to calculate average?
 $\text{Average} = \text{Sum} / \text{Number of elements}.$
6. What is the maximum size of the array used?
100 elements.
7. What happens if n is zero?
Division by zero occurs, so n should be greater than zero.
8. Can this program work for negative numbers?
Yes, it works for both positive and negative integers.

RESULT:-

The C program to read n integers, store them in an array, and find their sum and average was written, compiled, and executed successfully.

EXP.NO:9**PROGRAM TO READ N INTEGERS, STORE THEM IN AN ARRAY AND SEARCH FOR AN ELEMENT IN THE ARRAY USING THE LINEAR SEARCH ALGORITHM****AIM:-**

To write a program to read n integers, store them in an array, and search for a given element using the Linear Search algorithm.

LEARNING OUTCOMES:-

Understand how arrays work in programming.

Learn the concept of Linear Search.

Develop a program to search for an element in an array.

Analyze the time complexity of Linear Search.

DESCRIPTION:-

Linear Search is the simplest searching technique. In this method, each element of the array is compared one by one with the key element until the element is found or the list ends.

If the element is found, its position is displayed. Otherwise, a message stating "Element not found" is displayed.

Time Complexity:

- Best Case: $O(1)$
- Worst Case: $O(n)$

ALGORITHM:-

Step 1: Start

Step 2: Declare variables

- $n \rightarrow$ number of elements
- $arr[n] \rightarrow$ array to store integers
- $key \rightarrow$ element to search
- $i \rightarrow$ loop counter
- $found \rightarrow$ flag variable (initialize to 0)

Step 3: Read the value of n

Step 4: Read n integers and store them in the array arr

Step 5: Read the element to be searched (key)

Step 6: Set found = 0

Step 7: Repeat for i = 0 to n - 1

- If arr[i] == key
 - Set found = 1
 - Display "Element found at position i + 1"
 - Stop the loop

Step 8: After the loop, check:

- If found == 0
 - Display "Element not found"

Step 9: Stop

PROGRAM:-

```
#include <stdio.h>
```

```
int main() {  
    int n, i, key, flag = 0;  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
    int arr[n];  
    printf("Enter %d elements:\n", n);  
    for(i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }  
    printf("Enter element to search: ");  
    scanf("%d", &key);  
    for(i = 0; i < n; i++) {  
        if(arr[i] == key) {  
            printf("Element found at position %d", i + 1);  
            flag = 1;  
        }  
    }  
}
```

```
        break;
    }
}

if(flag == 0) {
    printf("Element not found");
}
return 0;
}
```

PROCEDURE:-

1. Start the program.
2. Enter the number of elements.
3. Enter the array elements.
4. Enter the element to be searched.
5. The program compares each element with the key.
6. If found, the position is displayed.
7. If not found, a message is displayed.
8. Stop the program.

OUTPUT:-

Enter number of elements: 5

Enter 5 elements:

10 20 30 40 50

Enter element to search: 30

Element found at position 3

Enter number of elements: 4

Enter 4 elements:

5 8 12 16

Enter element to search: 7

Element not found

MODEL VIVA QUESTIONS:-

1. What is Linear Search?
2. What is the time complexity of Linear Search?
3. What is the difference between Linear Search and Binary Search?
4. Can Linear Search be used on unsorted arrays?
5. What is the best case of Linear Search?
6. What is the worst case of Linear Search?
7. Why is Linear Search considered simple?

RESULT:-

The program to read n integers, store them in an array, and search for an element using the Linear Search algorithm was successfully executed and verified.

EXP.NO.10**BUBBLE SORT IMPLEMENTATION OF N INTEGERS****AIM:-**

To read n integers from the user, store them in an array, and sort the elements of the array in ascending order using the Bubble Sort algorithm.

LEARNING OUTCOMES:-

- Understand the concept of arrays.
- Learn how the Bubble Sort algorithm works.
- Develop a program to sort elements in ascending order.
- Improve logical thinking and programming skills.

DESCRIPTION:-

Bubble Sort is a simple sorting technique that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process continues until the array is completely sorted. In each pass, the largest unsorted element moves to its correct position, similar to a bubble rising to the surface.

ALGORITHM:-

1. Start.
2. Read the number of elements n .
3. Declare an array of size n .
4. Read n integers and store them in the array.
5. For $i = 0$ to $n-1$:
 For $j = 0$ to $n-i-2$
 :If $\text{array}[j] > \text{array}[j+1]$, swap the elements.
6. Display the sorted array.
7. Stop.

PROGRAM:-

```
#include <stdio.h>
int main() {
    int n, i, j, temp;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d integers:\n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    // Bubble Sort
    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    printf("Sorted array in ascending order:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

PROCEDURE:-

1. Compile the program using a C compiler.
2. Run the executable file.
3. Enter the number of elements.
4. Enter the integers when prompted.
5. The program sorts the elements using Bubble Sort.
6. The sorted array is displayed on the screen.

OUTPUT:-

```
Enter number of elements: 5
Enter 5 integers: 4 2 8 1 6
Sorted array in ascending order: 1 2 4 6 8
```

MODEL VIVA QUESTIONS:-

1. What is Bubble Sort?
2. Why is it called Bubble Sort?
3. What is the time complexity of Bubble Sort?
4. What is the space complexity of Bubble Sort?
5. Is Bubble Sort a stable sorting algorithm?
6. How many passes are required in Bubble Sort?
7. What is the best-case time complexity of Bubble Sort?
8. Can Bubble Sort sort in descending order? How?
9. What are the advantages and disadvantages of Bubble Sort?
10. Where is Bubble Sort used in real-life applications?

RESULT:-

Thus, the program to read n integers from the user, store them in an array, and sort the elements in ascending order using the Bubble Sort algorithm was successfully implemented and executed, and the desired sorted output was obtained.

EXP.11**PROGRAM TO CHECK WHETHER A GIVEN STRING IS A PALINDROME OR NOT****AIM:-**

To read a string (word), store it in an array, and check whether it is a palindrome or not.

LEARNING OUTCOMES:-

- Understand how to store a string in a character array.
- Learn how to compare characters using loops.
- Apply conditional statements to solve problems.
- Understand the concept of palindrome checking.

ALGORITHM:-

Step 1: Start

Step 2: Declare variables

- `str[100]` → character array
- `i`, `length`
- `flag = 0`

Step 3: Read the string

Step 4: Find the length of the string

Step 5: For `i = 0` to `length / 2`

- If `str[i] != str[length - i - 1]`
 - Set `flag = 1`
 - Break the loop

Step 6: If `flag == 0`

- Display "String is Palindrome"
- Else
- Display "String is Not Palindrome"

Step 7: Stop

PROGRAM:-

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[100],rstr[100];
    int i,j,ln;
    printf("Enter the string(word): ");
    scanf("%s",str);
    ln=strlen(str);// reversing the string
    for(i=ln-1,j=0;i>=0;i--,j++)
        rstr[j]=str[i];
    rstr[j]='\0';
    if(strcmp(rstr,str)==0)
        printf("Palindrome\n");
    else
        printf("Not Palindrome\n");
}
```

PROCEDURE:-

1. Start the program.
2. Declare a character array to store the string.
3. Accept the string from the user.
4. Calculate the length of the string.
5. Compare characters from the beginning and end.
6. If all characters match, print palindrome.
7. Otherwise, print not a palindrome.
8. Stop the program.

OUTPUT:-**Sample Output 1:**

Enter a string: MADAM
String is Palindrome

Sample Output 2:

Enter a string: HELLO
String is Not Palindrome

MODEL VIVA QUESTIONS:-

1. What is a palindrome?
2. What is a string in C?
3. Which header file is required for string functions?
4. What is the use of strlen()?
5. Why do we run the loop only till length / 2?
6. What is a character array?
7. What is the difference between scanf() and gets()?

RESULT:-

Thus, the program to read a string and check whether it is a palindrome or not was successfully executed and verified.

EXP.NO.12**CONCATENATE TWO STRINGS WITHOUT USING LIBRARY FUNCTIONS****AIM:-**

To read two strings ending with a \$ symbol, store them in arrays, and concatenate them without using any library functions.

LEARNING OUTCOMES:

- Understand how to work with arrays to store string data.
- Learn how to manually concatenate two strings using loops.
- Develop problem-solving skills without relying on built-in functions.
- Improve understanding of string manipulation in programming.

DESCRIPTION:-

This program takes two input strings that each end with a \$ symbol, stores them in arrays, and then concatenates them into a single string. The concatenation is done manually by iterating over each character of the input strings and appending them to a new string array.

ALGORITHM:-

1. Start
2. Declare two arrays to store the two input strings.
3. Read the first string and store it in the first array.
4. Read the second string and store it in the second array.
5. Initialize a third array to store the concatenated result.
6. Loop through the first array and copy each character into the result array.
7. Loop through the second array and copy each character into the result array.
8. Print the concatenated string.
9. End

PROGRAM:-

```
#include <stdio.h>

void concatenateStrings(char str1[], char str2[]) {

    int i = 0, j = 0;

    // Find the length of the first string

    while (str1[i] != '$') {
```

```
i++;  
  
}  
  
i++; // Skip the '$' symbol  
  
// Find the length of the second string  
while (str2[j] != '$') {  
    j++;  
}  
  
j++; // Skip the '$' symbol  
  
// Concatenate both strings into a new string  
char result[i + j + 1]; // +1 for null-terminator  
int k = 0;  
  
// Copy the first string into the result array  
for (int l = 0; l < i; l++) {  
    result[k++] = str1[l];  
}  
  
// Copy the second string into the result array  
for (int m = 0; m < j; m++) {  
    result[k++] = str2[m];  
}  
  
result[k] = '\0'; // Null-terminate the result string  
printf("Concatenated String: %s\n", result);  
}  
  
int main() {  
    char str1[100], str2[100];  
  
    // Read two strings ending with '$'
```

```
printf("Enter the first string ending with '$': ");  
  
scanf("%s", str1);  
  
printf("Enter the second string ending with '$': ");  
  
scanf("%s", str2);  
  
// Call function to concatenate strings  
  
concatenateStrings(str1, str2);  
  
return 0;  
  
}
```

PROCEDURE:-

1. Define two arrays (`str1` and `str2`) to hold the input strings.
2. Read both strings, ensuring that they end with the \$ symbol.
3. Iterate through both arrays, copying characters into a result array.
4. Output the concatenated string.

OUTPUT:-

Enter the first string ending with '\$': Hello\$

Enter the second string ending with '\$': World\$

Concatenated String: HelloWorld

MODEL VIVA QUESTIONS:-

1. What is the purpose of using arrays to store the strings?
2. Why is it important to stop reading the string when you encounter the \$ symbol?
3. How does the program handle the concatenation of the two strings manually?
4. What would happen if the \$ symbol was omitted in the input strings?
5. How do you ensure the result array is correctly terminated as a string?

RESULT:-

The program successfully concatenates two strings (ending with a \$ symbol) without using any library functions and prints the concatenated result.

EXP.NO.13**READ A STRING ENDING WITH A \$ SYMBOL, STORE IT IN AN ARRAY, AND COUNT THE VOWELS, CONSONANTS, AND SPACES****AIM:-**

To read a string that ends with a \$ symbol, store it in an array, and count the number of vowels, consonants, and spaces in it.

LEARNING OUTCOMES:-

- Understand how to read and store a string in an array.
- Learn to iterate through the string to count specific characters (vowels, consonants, and spaces).
- Improve knowledge of string manipulation and character checking in programming.
- Learn the concept of conditional checks for vowels, consonants, and spaces.

DESCRIPTION:-

This program will read a string input from the user, which ends with a \$ symbol. The string will be stored in an array, and the program will then iterate over the string to count how many vowels, consonants, and spaces are present. The program will output the counts of these characters separately.

ALGORITHM:-

1. Start
2. Declare an array to store the input string.
3. Initialize counters for vowels, consonants, and spaces.
4. Read the string from the user, ensuring it ends with the \$ symbol.
5. Iterate through the string:
 - If the character is a vowel (a, e, i, o, u), increment the vowel counter.
 - If the character is a consonant (alphabet letter but not a vowel), increment the consonant counter.
 - If the character is a space, increment the space counter.
6. Print the final counts of vowels, consonants, and spaces.
7. End

PROGRAM:-

```
#include <stdio.h>
#include <ctype.h> // For the isalpha function

void countCharacters(char str[]) {
    int vowels = 0, consonants = 0, spaces = 0;
    int i = 0;

    // Iterate through the string until '$' is encountered
    while (str[i] != '$' && str[i] != '\0') {
        char ch = tolower(str[i]); // Convert to lowercase for uniformity

        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
            vowels++;
        } else if (isalpha(ch)) {
            consonants++;
        } else if (ch == ' ') {
            spaces++;
        }
        i++;
    }

    // Print the counts of vowels, consonants, and spaces
    printf("Vowels: %d\n", vowels);
    printf("Consonants: %d\n", consonants);
    printf("Spaces: %d\n", spaces);
}

int main() {
    char str[100];

    // Read the string ending with '$'
    printf("Enter a string ending with '$': ");
    fgets(str, sizeof(str), stdin); // To allow spaces in the input

    // Call function to count vowels, consonants, and spaces
    countCharacters(str);

    return 0;
}
```

PROCEDURE:-

1. Define an array (`str`) to store the input string.
2. Initialize three counters: `vowels`, `consonants`, and `spaces` to 0.
3. Read the string from the user using `fgets( )` to handle spaces properly.
4. Iterate through the string until encountering the `$` symbol:
 - Check if the character is a vowel and increment the vowel counter.
 - Check if the character is a consonant and increment the consonant counter.
 - Check if the character is a space and increment the space counter.
5. Print the final count of vowels, consonants, and spaces.

OUTPUT:-

Enter a string ending with '\$': Hello World\$

Vowels: 3

Consonants: 8

Spaces: 1

MODEL VIVA QUESTIONS:-

1. How does the program differentiate between vowels, consonants, and spaces?
2. What is the purpose of using `tolower( )` in the program?
3. Why do you use `fgets( )` instead of `scanf( )` to read the string?
4. How does the program handle cases when the string contains uppercase letters or punctuation?
5. What would happen if the string does not end with a `$` symbol? How can we ensure this condition?

RESULT:-

The program successfully counts and displays the number of vowels, consonants, and spaces in a string that ends with a `$` symbol, using manual iteration and condition checks.

EXP.NO.14**ADDITION OF TWO DISTANCES USING STRUCTURES IN EUCLIDEAN SPACE****AIM:-**

To read two input values representing distances between two points in Euclidean space, store them in structure variables, and compute the sum of the two distance values.

LEARNING OUTCOMES:-

- Understand the concept of structures in C.
- Learn how to declare and use structure variables.
- Perform arithmetic operations using structure members.
- Develop a program to add two distance values.

DESCRIPTION:-

In Euclidean space, the distance between two points is represented as a numerical value. In this program, two distances are provided as input by the user. These distances are stored in structure variables. The program then adds the two distance values and displays the result. Structures help in organizing related data efficiently.

ALGORITHM:-

1. Start the program.
2. Define a structure to store a distance value.
3. Declare three structure variables (distance1, distance2, sum).
4. Read the first distance value from the user.
5. Read the second distance value from the user.
6. Add the two distance values and store the result in the sum variable.
7. Display the total distance.
8. Stop the program.

PROGRAM:-

```
#include <stdio.h>

struct distance {

    float value;

};
```

```
int main() {  
  
    struct distance d1, d2, sum;  
  
    printf("Enter first distance: ");  
  
    scanf("%f", &d1.value);  
  
    printf("Enter second distance: ");  
  
    scanf("%f", &d2.value);  
  
    sum.value = d1.value + d2.value;  
  
    printf("Sum of distances = %.2f\n", sum.value);  
  
    return 0;  
}
```

PROCEDURE:-

1. Open the C compiler (e.g., Turbo C / GCC).
2. Create a new program file.
3. Type the given program code.
4. Save the file with **.c** extension.
5. Compile the program.
6. Run the program.
7. Enter two distance values when prompted.
8. Observe the output displayed on the screen.

OUTPUT:-

Enter first distance: 12.5

Enter second distance: 7.3

Sum of distances = 19.80

MODEL VIVA QUESTIONS:-

1. What is a structure in C?
2. How do you declare a structure?
3. What is the difference between structure and union?
4. How do you access members of a structure?
5. Why are structures used in programming?
6. Can we perform arithmetic operations directly on structure variables?

RESULT:-

The program to read two distances using structures and calculate their sum was successfully executed and verified.

EXP.NO.15**READING AND PRINTING DATA OF N EMPLOYEES USING STRUCTURE****AIM:-**

To write a C program using structure to read and display the details of N employees such as Name, Employee ID, and Salary.

LEARNING OUTCOMES:-

- Understand the concept and usage of structures in C.
- Learn how to declare an array of structures.
- Develop skills to store and access multiple records using structures.
- Implement input and output operations for structured data.

DESCRIPTION:-

In many real-world applications, employee details such as Name, Employee ID, and Salary need to be stored and managed efficiently. Structures in C allow grouping of different data types under one name. In this program, a structure named employee is defined to store employee details. An array of structures is used to store data for multiple employees. The program reads the data of N employees and displays the entered details.

ALGORITHM:-

1. Start the program.
2. Define a structure employee with members: name, employee ID, and salary.
3. Declare an array of structures to store employee details.
4. Read the number of employees (N).
5. For each employee from 1 to N:
 - Read name.
 - Read employee ID.
 - Read salary.
6. Display the details of all employees.
7. Stop the program.

PROGRAM:-

```
#include <stdio.h>
struct Employee
{
int empid;
char name[50];
int salary;
} emp[50];
int main()
{ int n,i;
  system("clear");
  printf("Enter the number of employees\n");
  scanf("%d",&n);
  for(i=0;i<n;i++)
  {
    printf("Enter the employee details-%d\n",i+1);
    printf("Employee id:");
    scanf("%d",&emp[i].empid);
    getchar();
    printf("Employee name:");
    scanf("%[^\\n]",emp[i].name);
    printf("Employee salary:");
    scanf("%d",&emp[i].salary);
  }
  //printing the details
  printf("Employee Details\n");
  printf("Employee id---Employee name---Employee salary\n");
  for(i=0;i<n;i++)
    printf("%-15d %-15s %10d\n",emp[i].empid,emp[i].name,emp[i].salary);
}
```

PROCEDURE:-

1. Open the C compiler (Turbo C / GCC).
2. Create a new file and enter the program code.
3. Save the file with .C extension.
4. Compile the program.
5. Run the program.
6. Enter the number of employees and their details.
7. Verify the displayed output.

OUTPUT:-

Enter number of employees: 2

Employee 1

Enter Name: John

Enter Employee ID: 101

Enter Salary: 25000

Employee 2

Enter Name: Mary

Enter Employee ID: 102

Enter Salary: 30000

Employee Details:

Employee 1

Name: John

Employee ID: 101

Salary: 25000.00

Employee 2

Name: Mary

Employee ID: 102

Salary: 30000.00

MODEL VIVA QUESTIONS:-

1. What is a structure in C?
2. Why do we use an array of structures?
3. How do you access structure members?
4. What is the difference between structure and array?
5. Can a structure contain another structure?
6. What are the advantages of using structures?

RESULT:-

Thus, the program to read and print data of N employees using structure was successfully implemented and executed.

EXP.NO.16**DECLARE A UNION WITH 5 STRING VARIABLES AND DISPLAY ADDRESS****AIM:-**

To write a C program to declare a **union** containing five string variables (Name, House Name, City Name, State, and Pin Code) with user-defined size and to read and display the address of a person using a union variable.

LEARNING OUTCOME:-

After completing this experiment, students will be able to:

- Understand the difference between **structure and union** in C.
- Declare and define a union with string members.
- Access union members.
- Observe memory sharing behavior of union variables.

DESCRIPTION:-

A **union** in C is a user-defined data type similar to a structure. However, in a union, **all members share the same memory location**.

This means:

- Only one member can store a value at a time.
- The size of the union is equal to the size of its largest member.

In this program, we declare a union containing:

- Name
- House Name
- City Name
- State
- Pin Code

Since all are character arrays (strings), the union size will be equal to the largest array size.

ALGORITHM:-

1. Start the program.
2. Define a union with five character arrays (user-defined size).
3. Declare a union variable.
4. Read Name from the user and display it.
5. Read House Name and display it.
6. Read City Name and display it.
7. Read State and display it.
8. Read Pin Code and display it.
9. Stop the program.

PROGRAM:-

```
#include <stdio.h>
```

```
union Address {  
    char name[50];  
    char house[50];  
    char city[50];  
    char state[50];  
    char pincode[20];  
};
```

```
int main() {  
    union Address addr;  
  
    printf("Enter Name: ");  
    fgets(addr.name, sizeof(addr.name), stdin);  
    printf("Name: %s\n", addr.name);  
  
    printf("Enter House Name: ");  
    fgets(addr.house, sizeof(addr.house), stdin);  
    printf("House Name: %s\n", addr.house);  
  
    printf("Enter City Name: ");  
    fgets(addr.city, sizeof(addr.city), stdin);  
    printf("City Name: %s\n", addr.city);  
  
    printf("Enter State: ");  
    fgets(addr.state, sizeof(addr.state), stdin);  
    printf("State: %s\n", addr.state);  
}
```

```
printf("Enter Pin Code: ");  
fgets(addr.pincod, sizeof(addr.pincod), stdin);  
printf("Pin Code: %s\n", addr.pincod);  
  
return 0;  
}
```

PROCEDURE:-

1. Open Turbo C / GCC Compiler.
2. Create a new C file.
3. Type the program and save it with .c extension.
4. Compile the program.
5. Run the program.
6. Enter the required address details.
7. Observe the output.

OUTPUT:-

Enter Name: Rahul Sharma
Name: Rahul Sharma

Enter House Name: Green Villa
House Name: Green Villa

Enter City Name: Mumbai
City Name: Mumbai

Enter State: Maharashtra
State: Maharashtra

Enter Pin Code: 400001
Pin Code: 400001

MODEL VIVA QUESTIONS:-

1. What is a union in C?
2. What is the difference between structure and union?
3. How is memory allocated in a union?
4. Can we access all union members at the same time? Why?
5. What determines the size of a union?
6. Why is union memory-efficient compared to structure?

RESULT:-

The program was compiled and executed successfully.

EXP.NO:17**FACTORIAL OF A NATURAL NUMBER USING FUNCTIONS IN C****AIM:-**

To write a C program to find the factorial of a given natural number n using:

1. A non-recursive function
2. A recursive function

LEARNING OUTCOME:-

After completing this experiment, students will be able to:

- Understand the concept of functions in C.
- Differentiate between recursive and non-recursive functions.
- Implement factorial calculation using both methods.
- Analyze the difference in function calls and memory usage.

DESCRIPTION:-

The factorial of a natural number n is defined as:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1 \\ n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

Special case:

$$0! = 10! = 10! = 1$$

There are two ways to compute factorial:

1. Non-Recursive Method – Uses loops (e.g., `for` loop).
2. Recursive Method – A function calls itself until a base condition is met.

ALGORITHM:-**i) Non-Recursive Function**

1. Start
2. Read number n
3. Call function `fact(n)`
4. Inside function, initialize `fact = 1`
5. Use loop from 1 to n
6. Multiply and store result

7. Return factorial value
8. Print result
9. Stop

ii) Recursive Function

1. Start
2. Read number n
3. Call recursive function `fact(n)`
4. If `n == 0` or `n == 1`, return 1
5. Else return `n * fact(n-1)`
6. Print result
7. Stop.

PROGRAM:-

```
#include <stdio.h>

/* Non-recursive function */
long int fact_non_recursive(int n) {
    long int fact = 1;
    for(int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}

/* Recursive function */
long int fact_recursive(int n) {
    if(n == 0 || n == 1)
        return 1;
    else
        return n * fact_recursive(n - 1);
}

int main() {
    int n;

    printf("Enter a natural number: ");
    scanf("%d", &n);

    printf("\nFactorial using Non-Recursive Function = %ld",
```

```
fact_non_recursive(n));

printf("\nFactorial using Recursive Function = %ld",
fact_recursive(n));

return 0;
}
```

PROCEDURE:-

1. Open C compiler (Turbo C / GCC).
2. Create a new file and type the program.
3. Save the file with .c extension.
4. Compile the program.
5. Run the program.
6. Enter a natural number.
7. Observe the factorial calculated by both methods.

OUTPUT:-

Enter a natural number: 5

Factorial using Non-Recursive Function = 120

Factorial using Recursive Function = 120

MODEL VIVA QUESTIONS:-

1. What is recursion?
2. What is a base condition in recursion?
3. What happens if the base condition is not provided?
4. Which method uses more memory – recursive or non-recursive? Why?
5. Define factorial of 0.
6. What is function calling?

RESULT:-

The program to find the factorial of a given natural number using both non-recursive and recursive functions was executed successfully. Both methods produced the same correct result, verifying the correctness of the implementation.

EXP.NO.18**REVERSE A STRING USING A USER DEFINED FUNCTION****AIM:-**

To read a string (word), store it in an array, and obtain its reverse by using a user-defined function.

LEARNING OUTCOMES:-

- Understand how to store strings in an array.
- Learn how to use user defined functions in C.
- Understand the concept of passing strings to functions.
- Learn the logic for reversing a string.

DESCRIPTION:-

A string is a collection of characters stored in an array and terminated by a null character (\0). In this program, the user enters a string which is stored in a character array. A user defined function is created to reverse the string by iterating from the end of the array to the beginning and printing the characters in reverse order.

ALGORITHM:-

1. Start the program.
2. Declare a character array to store the string.
3. Read the string from the user.
4. Call the user defined function to reverse the string.
5. In the function, find the length of the string.
6. Print the characters from the last index to the first index.
7. Stop the program.

PROGRAM:-

```
#include <stdio.h>

#include <string.h>

void reverse(char str[])
{
    int i, len;
```

```
len = strlen(str);

printf("Reversed string: ");

for(i = len - 1; i >= 0; i--)
{
    printf("%c", str[i]);
}

}

int main()
{
    char str[100];

    printf("Enter a string: ");
    scanf("%s", str);
    reverse(str);
    return 0;
}
```

PROCEDURE:-

1. Open a C compiler.
2. Create a new program file.
3. Type the program for reversing a string using a user defined function.
4. Save the file with **.c** extension.
5. Compile the program.
6. Run the program.
7. Enter the string when prompted and observe the reversed output.

OUTPUT:-

Enter a string: Hello

Reversed string: olleH

MODEL VIVA QUESTIONS:-

1. What is a string in C?
2. What is a user defined function?
3. Which header file is used for string functions?
4. What does `strlen()` function do?
5. What is the use of `\0` in strings?
6. What is an array?

RESULT:-

Thus the program to read a string, store it in an array and obtain its reverse using a user defined function was executed successfully.

EXP.NO.19**PROGRAM FOR MATRIX OPERATIONS USING FUNCTIONS****AIM:-**

To write a menu driven program to perform matrix addition, matrix multiplication, and to find the transpose of a matrix using user defined functions to read, compute, and display the matrices.

LEARNING OUTCOMES:-

- Understand matrix representation using arrays.
- Learn to perform matrix addition and multiplication.
- Understand how to find the transpose of a matrix.
- Learn to implement menu driven programs.
- Learn the use of user defined functions in C.

DESCRIPTION:-

A matrix is a rectangular arrangement of elements in rows and columns. Various operations can be performed on matrices such as addition, multiplication and transpose. In this program, matrices are stored using two-dimensional arrays. A menu driven approach is used to allow the user to select the required operation. User defined functions are used to read matrices, add matrices, multiply matrices, find the transpose and display the matrices.

ALGORITHM:-

1. Start the program.
2. Declare matrices and variables for rows and columns.
3. Display the menu with options:
 - Matrix Addition
 - Matrix Multiplication
 - Transpose of Matrix
 - Exit
4. Read the user's choice.
5. Use a switch statement to perform the selected operation.
6. Call the function to read matrices.
7. Perform the selected matrix operation using functions.
8. Display the result using the display function.
9. Repeat the process until the user chooses Exit.
10. Stop the program.

PROGRAM:-

```
#include <stdio.h>

void readMatrix(int a[10][10], int r, int c)
{
    int i,j;
    for(i=0;i<r;i++)
    {
        for(j=0;j<c;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
}

void displayMatrix(int a[10][10], int r, int c)
{
    int i,j;
    for(i=0;i<r;i++)
    {
        for(j=0;j<c;j++)
        {
            printf("%d ",a[i][j]);
        }
        printf("\n");
    }
}

void addMatrix(int a[10][10], int b[10][10], int r, int c)
{
    int i,j,sum[10][10];

    for(i=0;i<r;i++)
    {
        for(j=0;j<c;j++)
        {
            sum[i][j]=a[i][j]+b[i][j];
        }
    }

    printf("Sum of matrices:\n");
    displayMatrix(sum,r,c);
}
```

```
}

void multiplyMatrix(int a[10][10], int b[10][10], int r1, int c1, int c2)
{
    int i,j,k,prod[10][10];

    for(i=0;i<r1;i++)
    {
        for(j=0;j<c2;j++)
        {
            prod[i][j]=0;
            for(k=0;k<c1;k++)
            {
                prod[i][j]+=a[i][k]*b[k][j];
            }
        }
    }

    printf("Product of matrices:\n");
    displayMatrix(prod,r1,c2);
}

void transposeMatrix(int a[10][10], int r, int c)
{
    int i,j,t[10][10];

    for(i=0;i<r;i++)
    {
        for(j=0;j<c;j++)
        {
            t[j][i]=a[i][j];
        }
    }

    printf("Transpose of matrix:\n");
    displayMatrix(t,c,r);
}

int main()
{
    int a[10][10],b[10][10];
    int r1,c1,r2,c2,choice;

    printf("Enter rows and columns of first matrix: ");
    scanf("%d%d",&r1,&c1);
```

```
printf("Enter first matrix:\n");
readMatrix(a,r1,c1);

printf("Enter rows and columns of second matrix: ");
scanf("%d%d",&r2,&c2);

printf("Enter second matrix:\n");
readMatrix(b,r2,c2);

printf("\n1.Matrix Addition\n2.Matrix Multiplication\n3.Transpose\n");
printf("Enter your choice: ");
scanf("%d",&choice);

switch(choice)
{
    case 1:
        if(r1==r2 && c1==c2)
            addMatrix(a,b,r1,c1);
        else
            printf("Addition not possible\n");
        break;

    case 2:
        if(c1==r2)
            multiplyMatrix(a,b,r1,c1,c2);
        else
            printf("Multiplication not possible\n");
        break;

    case 3:
        transposeMatrix(a,r1,c1);
        break;

    default:
        printf("Invalid choice");
}

return 0;
}
```

PROCEDURE:-

1. Open a C compiler or IDE.
2. Create a new file and type the program.
3. Save the file with **.c** extension.
4. Compile the program.
5. Run the program.
6. Enter the matrix elements when prompted.
7. Choose the required matrix operation from the menu.
8. Observe the result displayed.

OUTPUT:-

Enter rows and columns of first matrix: 2 2

Enter first matrix:

1 2

3 4

Enter rows and columns of second matrix: 2 2

Enter second matrix:

5 6

7 8

1.Matrix Addition

2.Matrix Multiplication

3.Transpose

Enter your choice: 1

Sum of matrices:

6 8

10 12

MODEL VIVA QUESTIONS:

1. What is a matrix in C programming?
2. What is a two dimensional array?
3. What is a menu driven program?
4. What condition is required for matrix addition?
5. What condition is required for matrix multiplication?
6. What is the transpose of a matrix?
7. Why are functions used in this program?

RESULT:-

Thus the menu driven program to perform matrix addition, multiplication and transpose using user defined functions was executed successfully.

EXP.NO.20**PROGRAM TO ADD AND SWAP TWO NUMBERS USING POINTERS****AIM:-**

To write a C program to add two numbers and swap two numbers using pointers and a user defined function.

LEARNING OUTCOMES:-

- Understand the concept of pointers in C.
- Learn how to access variables using pointer variables.
- Understand how to pass arguments to functions using pointers.
- Learn to swap two numbers using a user defined function.

DESCRIPTION:-

Pointers are variables that store the address of another variable. They are useful when we need to manipulate the original data inside functions. In this program, two numbers are added using pointers by accessing their memory addresses. A separate user defined function is used to swap two numbers by passing their addresses as arguments. This demonstrates the concept of call by reference using pointers.

ALGORITHM:-

1. Start the program.
2. Declare two integer variables and pointer variables.
3. Read two numbers from the user.
4. Store the addresses of the numbers in pointer variables.
5. Add the numbers using pointers and display the result.
6. Call the swap function by passing the addresses of the two numbers.
7. In the swap function, interchange the values using a temporary variable.
8. Display the swapped values.
9. Stop the program.

PROGRAM:-

```
#include <stdio.h>
```

```
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
```

```
*x = *y;
*y = temp;
}

int main()
{
    int a, b, sum;
    int *p, *q;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    p = &a;
    q = &b;

    sum = *p + *q;

    printf("Sum = %d\n", sum);

    swap(&a, &b);

    printf("After swapping:\n");
    printf("a = %d b = %d", a, b);

    return 0;
}
```

PROCEDURE:-

1. Open a C compiler or IDE.
2. Create a new program file.
3. Type the program for adding and swapping numbers using pointers.
4. Save the file with .c extension.
5. Compile the program.
6. Run the program. Enter two numbers when prompted.
7. Observe the sum and swapped values displayed.

OUTPUT:-

```
Enter two numbers: 5 10
Sum = 15
After swapping:
a = 10 b = 5
```

MODEL VIVA QUESTIONS:-

1. What is a pointer in C?
2. What does the & operator represent?
3. What does the * operator represent in pointers?
4. What is a user defined function?
5. What is call by reference?
6. Why are pointers used in swapping numbers?

RESULT:-

Thus the program to add two numbers and swap two numbers using pointers and a user defined function was executed successfully.

EXP.NO.21**INPUT AND PRINT THE ELEMENTS OF AN ARRAY USING POINTERS****AIM:-**

To write a C program to input and print the elements of an array using pointers.

LEARNING OUTCOME:-

- Understand the concept of pointers in C.
- Access array elements using pointer arithmetic.
- Input and display array elements through pointers.
- Differentiate between array indexing and pointer notation.

DESCRIPTION:-

A pointer is a variable that stores the address of another variable. In C, the name of an array represents the address of its first element. By assigning this address to a pointer, the array elements can be accessed using pointer arithmetic.

If `ptr` points to the first element of an array:

- `ptr + i` gives the address of the `i`th element.
- `*(ptr + i)` accesses the value stored at the `i`th element.

This method eliminates the need for array indexing while performing input and output operations.

ALGORITHM:-

1. Start the program.
2. Declare an integer array, pointer, and variable for the number of elements.
3. Read the size of the array.
4. Assign the base address of the array to the pointer.
5. Read the array elements using the pointer.
6. Display the array elements using the pointer.
7. Stop the program.

PROGRAM:-

```
#include <stdio.h>
int main()
{
    int arr[100];
    int n, i;
```

```
int * ptr = arr; // Pointer to arr[0]
printf("Enter size of array: ");
scanf("%d", &n);

printf("Enter elements in array:\n");
for (i = 0; i < n; i++)
{
    // (ptr + i) is equivalent to &arr[i]
    scanf("%d", (ptr + i));
}

printf("Array elements: \n");
for (i = 0; i < n; i++)
{
    // *(ptr + i) is equivalent to arr[i]
    printf("%d\n", *(ptr + i));
}
}
```

OUTPUT:-

Enter the number of elements: 5

Enter 5 elements:

10

20

30

40

50

Array elements are:

10 20 30 40 50

MODEL VIVA QUESTIONS:-

1. What is a pointer in C?
2. What does `ptr = arr;` mean?
3. What is the difference between `arr[i]` and `*(ptr + i)`?
4. Why is `scanf("%d", (ptr + i));` valid?
5. What does the `*` operator do?

RESULT:-

The C program to input and print the elements of an array using pointers was successfully executed, and the array elements were displayed correctly using pointer arithmetic.

EXP.NO.22**COMPUTE SUM OF THE ELEMENTS STORED IN AN ARRAY USING POINTERS AND USER DEFINED FUNCTIONS.****AIM:-**

To write a C program to compute the sum of the elements stored in an array using pointers and a user-defined function.

LEARNING OUTCOME:-

- Understand the concept of pointers and their relationship with arrays.
- Pass arrays to user-defined functions using pointers.
- Use pointer arithmetic to access array elements.
- Compute the sum of array elements using a user-defined function.
- Develop modular programs using functions and pointers in C.

DESCRIPTION:-

In C programming, the name of an array represents the address of its first element. A pointer can be used to access the elements of the array by using pointer arithmetic. A user-defined function improves modularity and code reusability by performing a specific task independently.

In this program, the base address of the array is passed to a user-defined function through a pointer. The function traverses the array using pointer arithmetic, adds all the elements, and returns the sum to the `main()` function, where it is displayed.

ALGORITHM:-

1. Start the program.
2. Declare an integer array, pointer, and variables for size and sum.
3. Read the number of elements in the array.
4. Read the array elements.
5. Pass the array and its size to the user-defined function.
6. Inside the function:
 - Initialize the sum to 0.
 - Traverse the array using pointer arithmetic.
 - Add each element to the sum.
 - Return the sum.
7. Display the sum returned by the function.
8. Stop the program.

PROGRAM:-

```
#include <stdio.h>

int sumArray(int *ptr, int n)
{
    int i, sum = 0;
    for(i = 0; i < n; i++)
    {
        sum = sum + *(ptr + i);
    }
    return sum;
}

int main()
{
    int arr[100], n, i, sum;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the array elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    sum = sumArray(arr, n);
    printf("Sum of array elements = %d\n", sum);
    return 0;
}
```

PROCEDURE:-

1. Create a user-defined function `sumArray( )` that accepts a pointer and the size of the array.
2. Initialize the sum variable to zero.
3. Traverse the array using the pointer.
4. Add each element to the sum using pointer arithmetic `*(ptr + i)`.
5. Return the computed sum.
6. In the `main( )` function, read the array size and elements.
7. Call the `sumArray( )` function.
8. Display the returned sum.

OUTPUT:-

Sample Output:

Enter the number of elements: 5

Enter the array elements:

10

20

30

40

50

Sum of array elements = 150

MODEL VIVA QUESTIONS:-

1. What is a pointer in C?
2. What is pointer arithmetic?
3. Why are pointers used with arrays?
4. What is a user-defined function?
5. How is an array passed to a function?
6. What does `*(ptr + i)` represent?
7. Can an array name be used as a pointer? Explain.
8. What is the return type of the `sumArray( )` function?
9. What is the difference between `arr[i]` and `*(arr + i)`?
10. What are the advantages of using pointers?

RESULT:-

The program was executed successfully and the sum of the array elements was computed using pointers and a user-defined function.

EXP.NO.23**CREATE A FILE, WRITE DATA, READ AND DISPLAY FILE CONTENTS, APPEND NEW DATA, AND DISPLAY THE UPDATED FILE CONTENTS USING C FILE HANDLING****AIM:-**

To create a file and perform write, read, and append operations using file handling functions in C.

LEARNING OUTCOMES:-

- Understand the concept of file handling in C.
- Learn to create and open a file.
- Write data to a file using file handling functions.
- Read data from a file and display it on the console.
- Append new data to an existing file.
- Understand the use of different file modes (w, r, and a).

DESCRIPTION:-

File handling in C enables data to be stored permanently in files. The stdio.h library provides functions such as fopen(), fprintf(), fscanf(), fgets(), fputs(), and fclose() for file operations. In this program, a file is created, data is written to it, the stored data is read and displayed, additional data is appended, and finally the updated file contents are displayed on the console.

ALGORITHM:-

1. Start.
2. Open a file in write (w) mode.
3. Accept data from the user and write it to the file.
4. Close the file.
5. Open the same file in read (r) mode.
6. Read the contents of the file and display them on the console.
7. Close the file.
8. Open the file in append (a) mode.
9. Accept additional data from the user and append it to the file.
10. Close the file.
11. Open the file again in read (r) mode.
12. Read and display the updated file contents.
13. Close the file.
14. Stop.

PROGRAM:-

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
    char str[100];
```

```
    /* Write to file */
```

```
    fp = fopen("sample.txt", "w");
```

```
    printf("Enter data to write into file: ");
```

```
    gets(str);
```

```
    fprintf(fp, "%s\n", str);
```

```
    fclose(fp);
```

```
    /* Read from file */
```

```
    fp = fopen("sample.txt", "r");
```

```
    printf("\nContents of the file:\n");
```

```
    while(fgets(str, sizeof(str), fp) != NULL)
```

```
    {
```

```
        printf("%s", str);
```

```
    }
```

```
    fclose(fp);
```

```
    /* Append to file */
```

```
    fp = fopen("sample.txt", "a");
```

```
    printf("\nEnter data to append: ");
```

```
    getchar();
```

```
    gets(str);
```

```
    fprintf(fp, "%s\n", str);
```

```
    fclose(fp);
```

```
    /* Display updated contents */
```

```
    fp = fopen("sample.txt", "r");
```

```
    printf("\nUpdated file contents:\n");
```

```
    while(fgets(str, sizeof(str), fp) != NULL)
```

```
    {
```

```
        printf("%s", str);
```

```
    }
```

```
    fclose(fp);
```

```
    return 0;}
```

PROCEDURE:-

1. Create a file pointer using the FILE data type.
2. Open the file in write mode using fopen().
3. Write user input to the file using fprintf().
4. Close the file using fclose().
5. Open the file in read mode.
6. Read the contents using fgets() and display them on the console.
7. Close the file.
8. Open the file in append mode.
9. Append additional data using fprintf().
10. Close the file.
11. Reopen the file in read mode.
12. Display the updated contents.
13. Close the file.

OUTPUT:-

Enter data to write into file: Welcome to C Programming

Contents of the file:

Welcome to C Programming

Enter data to append: File Handling in C

Updated file contents:

Welcome to C Programming

File Handling in C

MODEL VIVA QUESTIONS:-

1. What is file handling in C?
2. What is the purpose of the FILE pointer?
3. Which header file is required for file handling?
4. What are the different file opening modes in C?
5. What is the difference between w and a modes?
6. Why is fclose() used?
7. What is the purpose of fgets()?
8. What is the difference between fprintf() and printf()?
9. What happens if the file does not exist while opening it in write mode?
10. Why is file handling important?

RESULT:-

The program was executed successfully. A file was created, data was written to it, the contents were read and displayed, new data was appended, and the updated file contents were displayed successfully.

EXP.NO.24**COUNT THE NUMBER OF CHARACTERS, WORDS, AND LINES IN A TEXT FILE****AIM:-**

To write a C program to open a text input file, count the number of characters, words, and lines present in it, and store the results in an output file.

LEARNING OUTCOMES:-

- Understand file handling in C using `fopen()`, `fgetc()`, `fprintf()`, and `fclose()`.
- Learn to read data from a text file character by character.
- Count the number of characters, words, and lines in a file.
- Write the processed results into an output file.
- Handle file opening errors effectively.

DESCRIPTION:-

File handling in C allows data to be stored permanently in files. In this program, the input file (input.txt) is opened in read mode. The program reads each character one by one using `fgetc()`. It counts:

- Characters by counting every character read.
- Words by detecting transitions from whitespace to non-whitespace characters.
- Lines by counting newline (`\n`) characters.

The results are then written into an output file (output.txt) using `fprintf()`.

ALGORITHM:-

1. Start.
2. Open the input file in read mode.
3. If the file cannot be opened, display an error message and stop.
4. Initialize character, word, and line counters to zero.
5. Read one character at a time until the end of the file.
6. Increment the character count.
7. If the character is a newline (`\n`), increment the line count.
8. If the character starts a new word, increment the word count.
9. Close the input file.
10. Open the output file in write mode.
11. Write the character count, word count, and line count into the output file.
12. Close the output file.
13. Stop.

PROGRAM:-

```
#include <stdio.h>
#include <ctype.h>

int main()
{
    FILE *input, *output;
    char ch;
    int characters = 0, words = 0, lines = 0, inWord = 0;

    input = fopen("input.txt", "r");

    if (input == NULL)
    {
        printf("Error opening input file.\n");
        return 1;
    }

    while ((ch = fgetc(input)) != EOF)
    {
        characters++;

        if (ch == '\n')
            lines++;

        if (isspace(ch))
            inWord = 0;
        else if (!inWord)
        {
            words++;
            inWord = 1;
        }
    }

    fclose(input);

    output = fopen("output.txt", "w");

    if (output == NULL)
    {
        printf("Error opening output file.\n");
    }
}
```

```
    return 1;
}

fprintf(output, "Number of Characters: %d\n", characters);
fprintf(output, "Number of Words: %d\n", words);
fprintf(output, "Number of Lines: %d\n", lines);

fclose(output);

printf("Results stored in output.txt\n");

return 0;
}
```

PROCEDURE:-

1. Open any C compiler (Turbo C, CodeBlocks, Dev-C++, or GCC).
2. Create a text file named input.txt and enter the required text.
3. Save the C program and compile it.
4. Execute the program.
5. The program reads the contents of input.txt.
6. It counts the number of characters, words, and lines.
7. The results are written into output.txt.
8. Open output.txt to verify the output.

OUTPUT:-

input.txt

Hello World

Welcome to C Programming.

output.txt

Number of Characters: 37

Number of Words: 6

Number of Lines: 1

MODEL VIVA QUESTIONS:-

1. What is file handling in C?
2. What are the different file opening modes in C?
3. What is the purpose of fopen()?
4. Why is fclose() necessary?
5. What is the use of fgetc()?
6. What is the purpose of fprintf()?
7. How are words counted in this program?
8. Which header file contains the isspace() function?
9. What is the significance of the EOF value?
10. What happens if the input file does not exist?

RESULT:-

The C program to open a text input file, count the number of characters, words, and lines, and store the results in an output file was successfully implemented and executed.